

WEEK12-PRACTICAL

Task1:

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
UW PICO 5.89 File: producer_consumer.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5
#define ITEMS_PER_PRODUCER 20

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++)
    {
        /*
         * Wait for an empty slot.
         */
        sem_wait(&empty);

        /*
         * Protect buffer and index.
         */
        pthread_mutex_lock(&mutex);

        buffer[in] = i;

        printf("Producer produced: %d at index %d\n",
              i, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        /*
         * One more item is available.
         */
        sem_post(&full);

        usleep(10000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++)
    {
        /*
         * Wait for an item to be available.
         */
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumer consumed: %d at index %d\n",
              item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}
```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell

```
UW PICO 5.09 File: producer_consumer.c Modified
/* Protect buffer and index.
 */
pthread_mutex_lock(&mutex);
buffer[in] = i;
printf("Producer produced: %d at index %d\n",
      i, in);
in = (in + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex);
/*
 * One more item is available.
 */
sem_post(&full);
usleep(10000);
}
return NULL;
}
void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++)
    {
        /*
         * Wait until an item exists.
         */
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer consumed: %d from index %d\n",
              item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        /*
         * One more empty slot is available.
         */
        sem_post(&empty);
        usleep(15000);
    }
    return NULL;
}
int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;
    Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
    Exit          Justify       Where is       Next Pg      UnCut Text    To Spell

```

```
UW PICO 5.09 File: producer_consumer.c Modified
/*
sem_wait(&full);
pthread_mutex_lock(&mutex);
int item = buffer[out];
printf("Consumer consumed: %d from index %d\n",
      item, out);
out = (out + 1) % BUFFER_SIZE;
pthread_mutex_unlock(&mutex);
/*
 * One more empty slot is available.
 */
sem_post(&empty);
usleep(15000);
}
return NULL;
}
int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;
    /*
     * Initially all buffer slots are empty.
     */
    sem_init(&empty, 0, BUFFER_SIZE);
    /*
     * Initially no items exist.
     */
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    printf("Producer-Consumer Problem\n");
    printf("Buffer size: %d\n", BUFFER_SIZE);
    clock_t start = clock();
    pthread_create(
        &producer_thread,
        NULL,
        producer,
        NULL
    );
    pthread_create(
        &consumer_thread,
        NULL,
        consumer,
        NULL
    );
    Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
    Exit          Justify       Where is       Next Pg      UnCut Text    To Spell

```

```
UW PICO 5.09 File: producer_consumer.c Modified
/*
 * Initially all buffer slots are empty.
 */
sem_init(&empty, 0, BUFFER_SIZE);

/*
 * Initially no items exist.
 */
sem_init(&full, 0, 0);

pthread_mutex_init(&mutex, NULL);

printf("Producer-Consumer Problem\n");
printf("Buffer size: %d\n", BUFFER_SIZE);

clock_t start = clock();

pthread_create(
    &producer_thread,
    NULL,
    producer,
    NULL
);

pthread_create(
    &consumer_thread,
    NULL,
    consumer,
    NULL
);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_t end = clock();

double elapsed =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("\nAll items produced and consumed\n");
printf("CPU time: %.2f seconds\n", elapsed);

sem_destroy(&empty);
sem_destroy(&full);

pthread_mutex_destroy(&mutex);

return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text At To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air os % mkdir week12pract
vishalkarnati@VISHALS-MacBook-Air os % cd week12pract
vishalkarnati@VISHALS-MacBook-Air week12pract % nano producer_consumer.c
vishalkarnati@VISHALS-MacBook-Air week12pract % gcc producer_consumer.c -o producer_consumer -pthread
producer_consumer.c:94:5: warning: 'sem_init' is deprecated [-Wdeprecated-declarations]
 94 |     sem_init(&empty, 0, BUFFER_SIZE);
    |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:55:42: note: 'sem_init' has been explicitly marked deprecated here
 55 | int sem_init(sem_t *, int, unsigned int) __deprecated;
    |
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
 223 | #define __deprecated __attribute__((__deprecated__))
    |
producer_consumer.c:99:5: warning: 'sem_init' is deprecated [-Wdeprecated-declarations]
 99 |     sem_init(&full, 0, 0);
    |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:55:42: note: 'sem_init' has been explicitly marked deprecated here
 55 | int sem_init(sem_t *, int, unsigned int) __deprecated;
    |
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
 223 | #define __deprecated __attribute__((__deprecated__))
    |
producer_consumer.c:133:5: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
133 |     sem_destroy(&empty);
    |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
 53 | int sem_destroy(sem_t *) __deprecated;
    |
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
 223 | #define __deprecated __attribute__((__deprecated__))
    |
producer_consumer.c:134:5: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
134 |     sem_destroy(&full);
    |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
 53 | int sem_destroy(sem_t *) __deprecated;
    |
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '__deprecated'
 223 | #define __deprecated __attribute__((__deprecated__))
    |
4 warnings generated.
vishalkarnati@VISHALS-MacBook-Air week12pract % ./producer_consumer
Producer-Consumer Problem
Buffer size: 5

Producer produced: 1 at index 0
Consumer consumed: 1 from index 0
Producer produced: 2 at index 1
Consumer consumed: 2 from index 1
Producer produced: 3 at index 2
Consumer consumed: 3 from index 2
Producer produced: 4 at index 3
Producer produced: 5 at index 4
Consumer consumed: 4 from index 3
Producer produced: 6 at index 0
Consumer consumed: 5 from index 4
Producer produced: 7 at index 1
Consumer consumed: 6 from index 0
Producer produced: 8 at index 2
Producer produced: 9 at index 3
Consumer consumed: 7 from index 1
Producer produced: 10 at index 4
Consumer consumed: 8 from index 2
Producer produced: 11 at index 0
```

```

/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
53 | int sem_destroy(sem_t *) __deprecated;
    |
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '___deprecated'
223 | #define ___deprecated __attribute__((__deprecated__))
    |
producer_consumer.c:134:8: warning: 'sem_destroy' is deprecated [-Wdeprecated-declarations]
134 |     sem_destroy(&full);
    |     ^
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/semaphore.h:53:26: note: 'sem_destroy' has been explicitly marked deprecated here
53 | int sem_destroy(sem_t *) __deprecated;
    |
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/sys/cdefs.h:223:40: note: expanded from macro '___deprecated'
223 | #define ___deprecated __attribute__((__deprecated__))
    |
4 warnings generated.
vishalkarnati@VISHALS-MacBook-Air week12pract % ./producer_consumer
Producer-Consumer Problem
Buffer size: 5

Producer produced: 1 at index 0
Consumer consumed: 1 from index 0
Producer produced: 2 at index 1
Consumer consumed: 2 from index 1
Producer produced: 3 at index 2
Consumer consumed: 3 from index 2
Producer produced: 4 at index 3
Consumer consumed: 4 from index 3
Producer produced: 5 at index 4
Consumer consumed: 5 from index 4
Producer produced: 6 at index 0
Consumer consumed: 6 from index 0
Producer produced: 7 at index 1
Consumer consumed: 7 from index 1
Producer produced: 8 at index 2
Consumer consumed: 8 from index 2
Producer produced: 9 at index 3
Consumer consumed: 9 from index 3
Producer produced: 10 at index 4
Consumer consumed: 10 from index 4
Producer produced: 11 at index 0
Consumer consumed: 11 from index 0
Producer produced: 12 at index 1
Consumer consumed: 12 from index 1
Producer produced: 13 at index 2
Consumer consumed: 13 from index 2
Producer produced: 14 at index 3
Consumer consumed: 14 from index 3
Producer produced: 15 at index 4
Consumer consumed: 15 from index 4
Producer produced: 16 at index 0
Consumer consumed: 16 from index 0
Producer produced: 17 at index 1
Consumer consumed: 17 from index 1
Producer produced: 18 at index 2
Consumer consumed: 18 from index 2
Producer produced: 19 at index 3
Consumer consumed: 19 from index 3
Producer produced: 20 at index 4
Consumer consumed: 20 from index 4

All items produced and consumed.
CPU time: 0.001794 seconds
vishalkarnati@VISHALS-MacBook-Air week12pract %

```

Task2:

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

PART A — Deliberate Deadlock:

```
UW PICO 5.09 File: deadlock.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Resource 1 locked\n");
    sleep(1);
    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Resource 2 locked\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Resource 2 locked\n");
    sleep(1);
    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Resource 1 locked\n");
    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
    return NULL;
}

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Program completed.\n");
    return 0;
}
```

```
UW PICO 5.09 File: deadlock.c Modified
void *thread2_function(void *arg)
{
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Resource 2 locked\n");
    sleep(1);
    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Resource 1 locked\n");
    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
    return NULL;
}

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(
        &thread1,
        NULL,
        thread1_function,
        NULL
    );

    pthread_create(
        &thread2,
        NULL,
        thread2_function,
        NULL
    );

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed.\n");
    return 0;
}
```

```

vishalkarnati@VISHALS-MacBook-Air: week12pract % nano deadlock.c
vishalkarnati@VISHALS-MacBook-Air: week12pract % gcc deadlock.c -o deadlock -pthread
vishalkarnati@VISHALS-MacBook-Air: week12pract % ./deadlock
Thread 1: locking Resource 1
Thread 1: Resource 1 locked
Thread 2: locking Resource 2
Thread 2: Resource 2 locked
Thread 1: waiting for Resource 2
Thread 2: waiting for Resource 1

```

PART B — Prevent Deadlock Using Resource Ordering:

```

UN PICO 5.09
File: deadlock_prevented.c
Modified

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Resource 1 locked\n");
    sleep(1);
    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Resource 2 locked\n");
    printf("Thread 1: using both resources\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    printf("Thread 1: resources released\n");
    return NULL;
}

void *thread2_function(void *arg)
{
    /*
     * IMPORTANT:
     * Thread 2 also locks Resource 1 FIRST.
     */
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Resource 1 locked\n");
    sleep(1);
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Resource 2 locked\n");
    printf("Thread 2: using both resources\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
}

```

```
UW PICO 5.09 File: deadlock_prevented.c Modified
}
return NULL;
}
void *thread2_function(void *arg)
{
    /*
     * IMPORTANT:
     * Thread 2 also locks Resource 1 FIRST.
     */
    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Resource 1 locked\n");
    sleep(1);
    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Resource 2 locked\n");
    printf("Thread 2: using both resources\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    printf("Thread 2: resources released\n");
    return NULL;
}
int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(
        &thread1,
        NULL,
        thread1_function,
        NULL
    );

    pthread_create(
        &thread2,
        NULL,
        thread2_function,
        NULL
    );

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
}
[Get Help] [WriteOut] [Read File] [Prev Pg] [Cut Text] [Cur Pos]
[Exit] [Justify] [Where is] [Next Pg] [UnCut Text] [To Spell]
```

```
UW PICO 5.09 File: deadlock_prevented.c Modified
printf("Thread 2: resources released\n");
return NULL;
}
int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(
        &thread1,
        NULL,
        thread1_function,
        NULL
    );

    pthread_create(
        &thread2,
        NULL,
        thread2_function,
        NULL
    );

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nNo deadlock occurred.\n");
    return 0;
}
[Get Help] [WriteOut] [Read File] [Prev Pg] [Cut Text] [Cur Pos]
[Exit] [Justify] [Where is] [Next Pg] [UnCut Text] [To Spell]
```

```
vishalkarnati@VISHALS-MacBook-Air week12pract % nano deadlock_prevented.c
vishalkarnati@VISHALS-MacBook-Air week12pract % gcc deadlock_prevented.c -o deadlock_prevented -pthread
vishalkarnati@VISHALS-MacBook-Air week12pract % ./deadlock_prevented
Thread 1: locking Resource 1
Thread 1: Resource 1 locked
Thread 2: locking Resource 1
Thread 1: locking Resource 2
Thread 1: Resource 2 locked
Thread 1: using both resources
Thread 1: resources released
Thread 2: Resource 1 locked
Thread 2: locking Resource 2
Thread 2: Resource 2 locked
Thread 2: using both resources
Thread 2: resources released

No deadlock occurred.
vishalkarnati@VISHALS-MacBook-Air week12pract %
```